

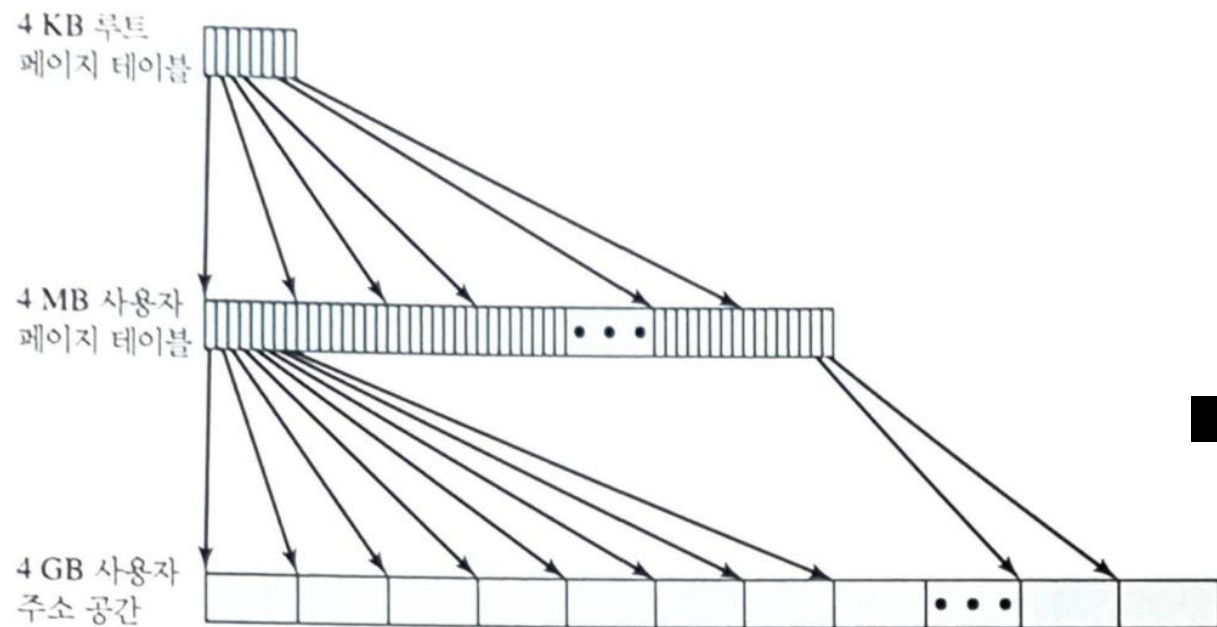
Paging & TLB

문승은

목차

1. Multi-Level Paging
2. TLB
3. TLB와 캐시의 동작
4. 문맥교환과 TLB Flush
5. 멀티코어와 TLB

Multi-Level Paging



4KB 크기 페이지
4GB 사용자 주소 공간
페이지 테이블 항목에 사상

➡ 4MB(2^{20})크기의 페이지 테이블

모든 프로세스마다 4MB 크기의
"연속된"메모리 할당은 낭비

그림 8.3 2-단계 계층적 페이지테이블

Why use multi-level Paging?

1. 메모리 오버헤드 감소

각 레벨에 포함되는 항목수가 적어 페이지 테이블을 저장하는데 필요한 메모리가 감소

2. 더 빠른 테이블 조회

레벨 당 엔트리 수가 적어 조회시간 단축

3. 유연성

동적으로 페이지 테이블을 조정가능

4. 쓰이지 않는 메모리 사용 X

32-bit

2^{32} (4GB)의 주소공간

PTE 크기 4byte

페이지 크기가 4KB일 때, (2^{12}) 오프셋 = 12bits로 표현

➔ 페이지 개수는 2^{20} 개

➔ 각 프로세스마다 $(2^{20} * 2^2)$ 4MB의 페이지 테이블

* 페이지 크기를 늘려 개수를 줄인다고 해도 내부 단편화가 심해짐.

64-bit

2^{48} 의 주소공간 (48bit 사용)

PTE크기는 64비트 시스템에서 보통 8byte

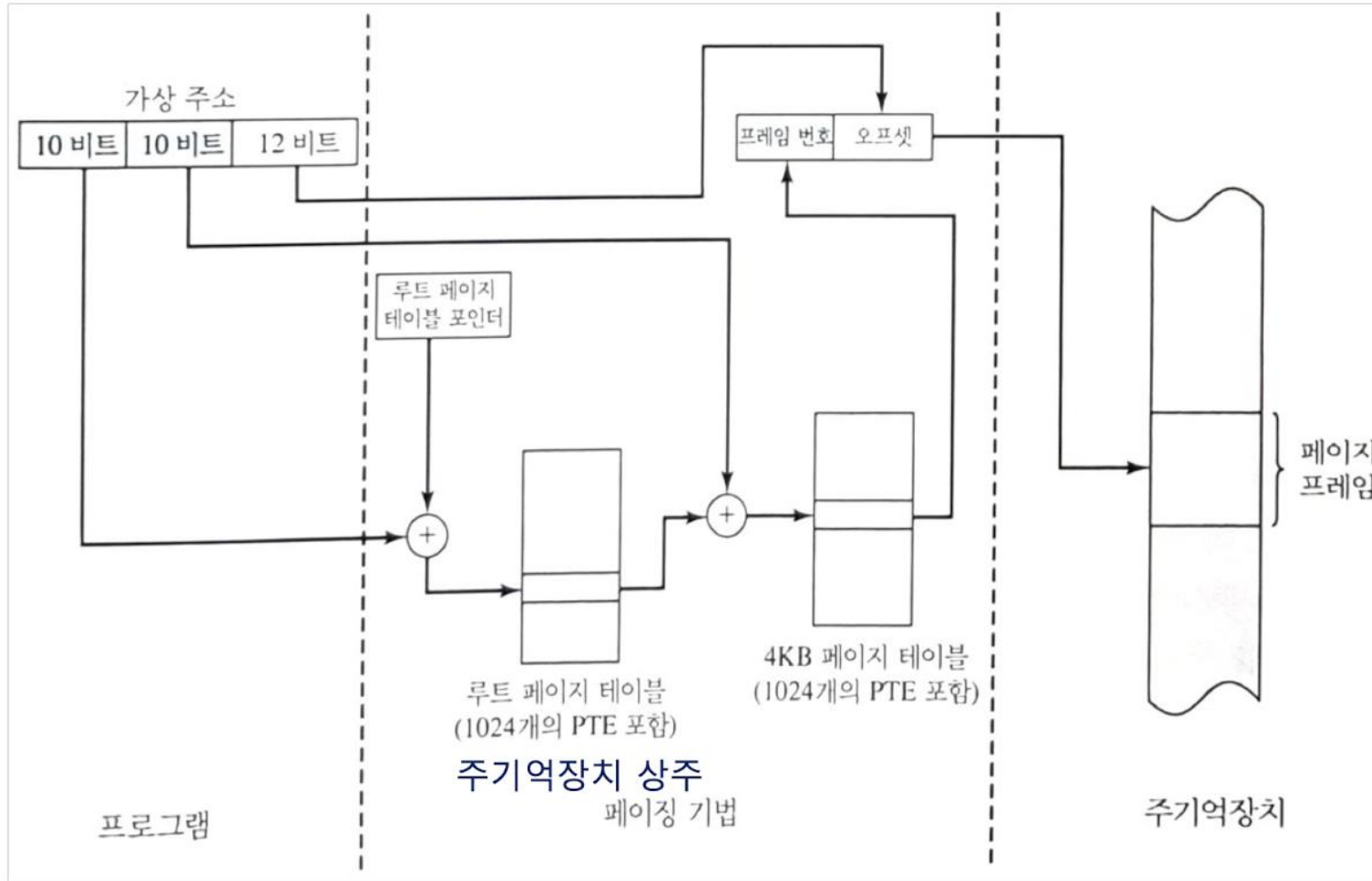
페이지 크기가 4KB일 때, 오프셋 = 12bits로 표현

→ 페이지 개수 = $2^{48}/2^{12} = 2^{36}$

→ 페이지 테이블 크기: $2^{36} * 2^3 = 2^{39}\text{byte}(512\text{GB})$

→ 64bit로 하면 32PB(WoW) (그래서 48비트만 사용)

2-level Paging, 32 bit



*참고

- 상위 레벨 페이지 테이블 항목은 하위 레벨 페이지 테이블을 가리키는 포인터
- 최종 레벨 페이지 테이블의 항목이 실제 프레임 정보를 저장

3-level Paging: Linux

- 페이지 디렉토리 + 페이지 중간 디렉토리 + 페이지 테이블

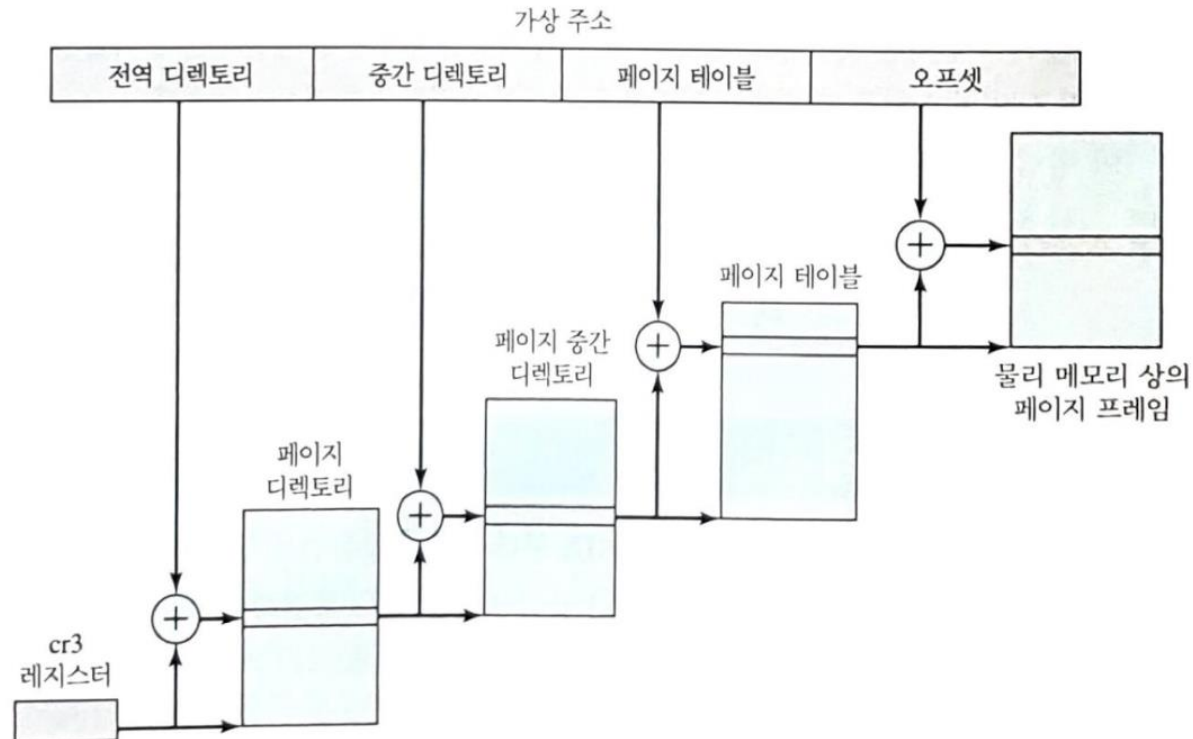


그림 8.23 Linux 가상메모리 방식에서의 주소변환

4-level paging

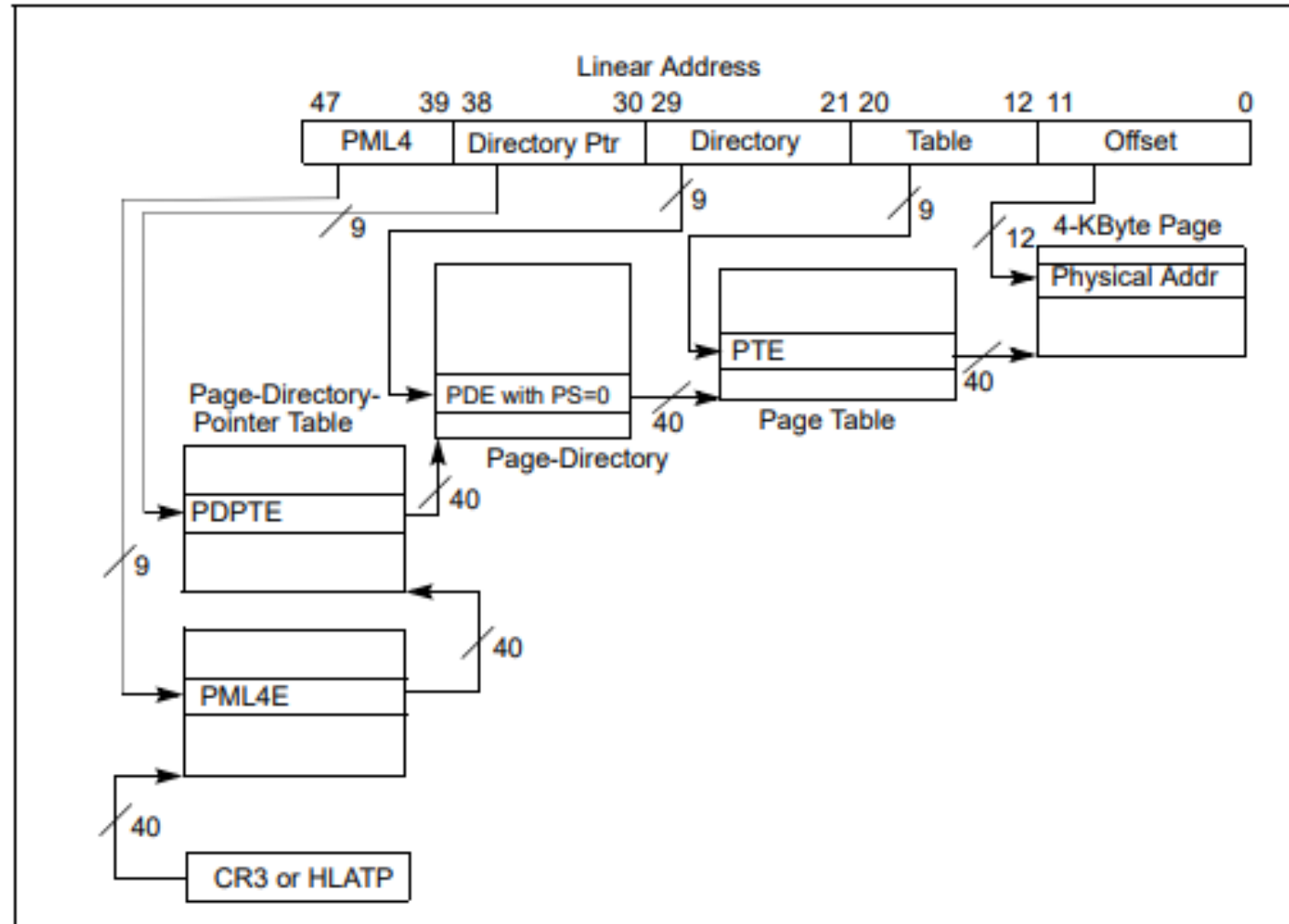


Figure 4-8. Linear-Address Translation to a 4-KByte Page using 4-Level Paging

5-level paging

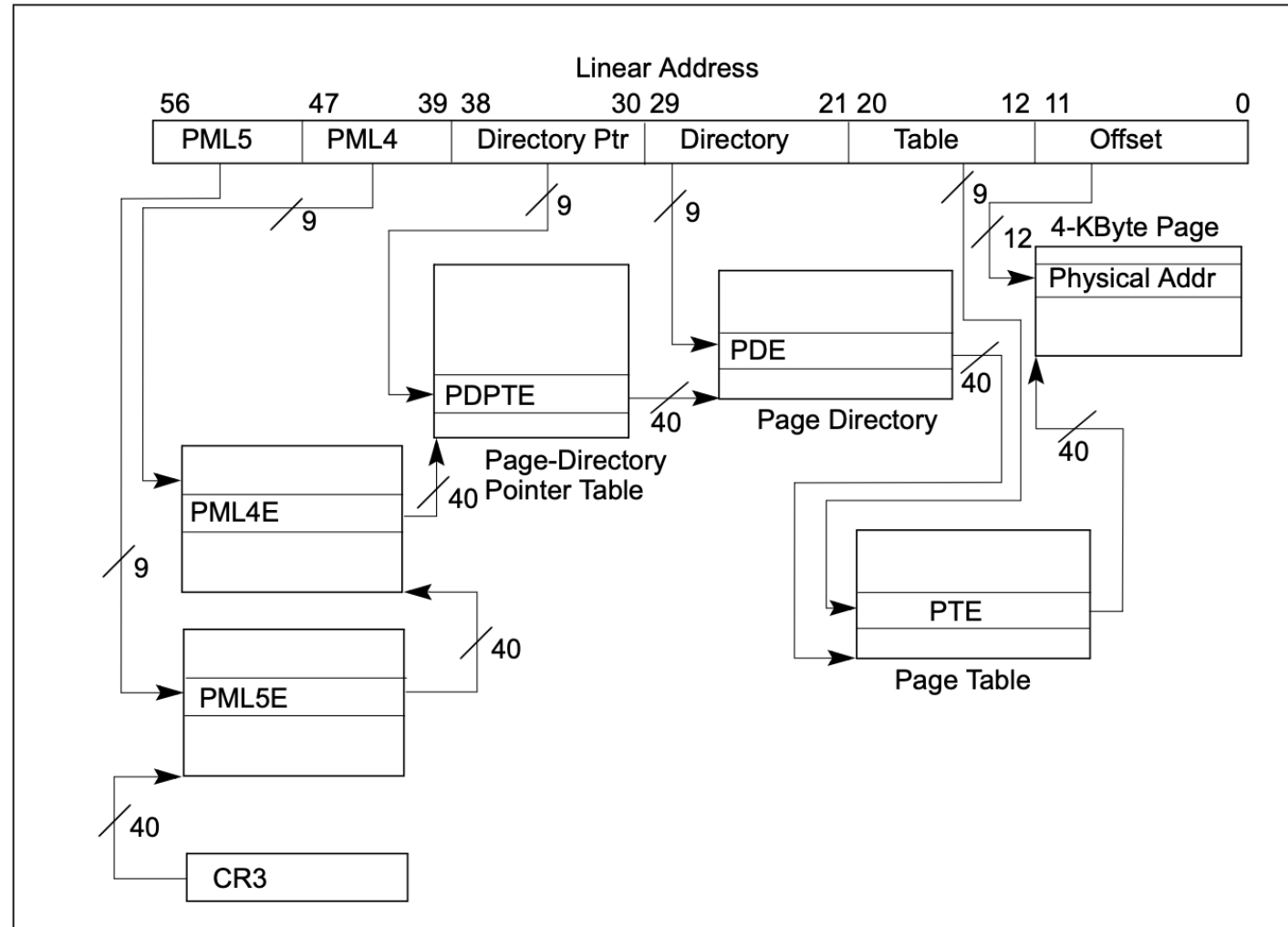


Figure 2-1. Linear-Address Translation Using 5-Level Paging

Trade-off

1. 복잡성 증가
2. 오버헤드 증가(각 단계 탐색) → TLB
3. 단편화 -> 페이지 테이블 항목이 연속적이지 않을 수 있음

Where to place the page table?

보통 메인 메모리에 있다..

Why not store Page Table in Registers?

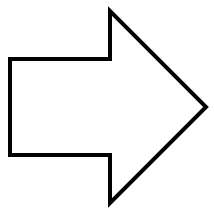
1. 레지스터의 사이즈는 한계가 있음 -> 큰 페이지 테이블을 저장할 수 없어요...
2. 그래서 메인 메모리
3. 그럼 느리지 않나..? -> TLB

TLB(Translation Lookaside Buffer)

가상메모리 참조는 총 두 번의 물리메모리 참조

1. 페이지테이블 항목을 참조
2. 요구된 데이터를 접근하기 위함

2배의 메모리 접근 시간 !



특수 고속 캐시인 **TLB** 사용

최근 참조된 페이지 테이블 항목들을 유지할 수 있도록,,,

MMU(Memory Management Unit)



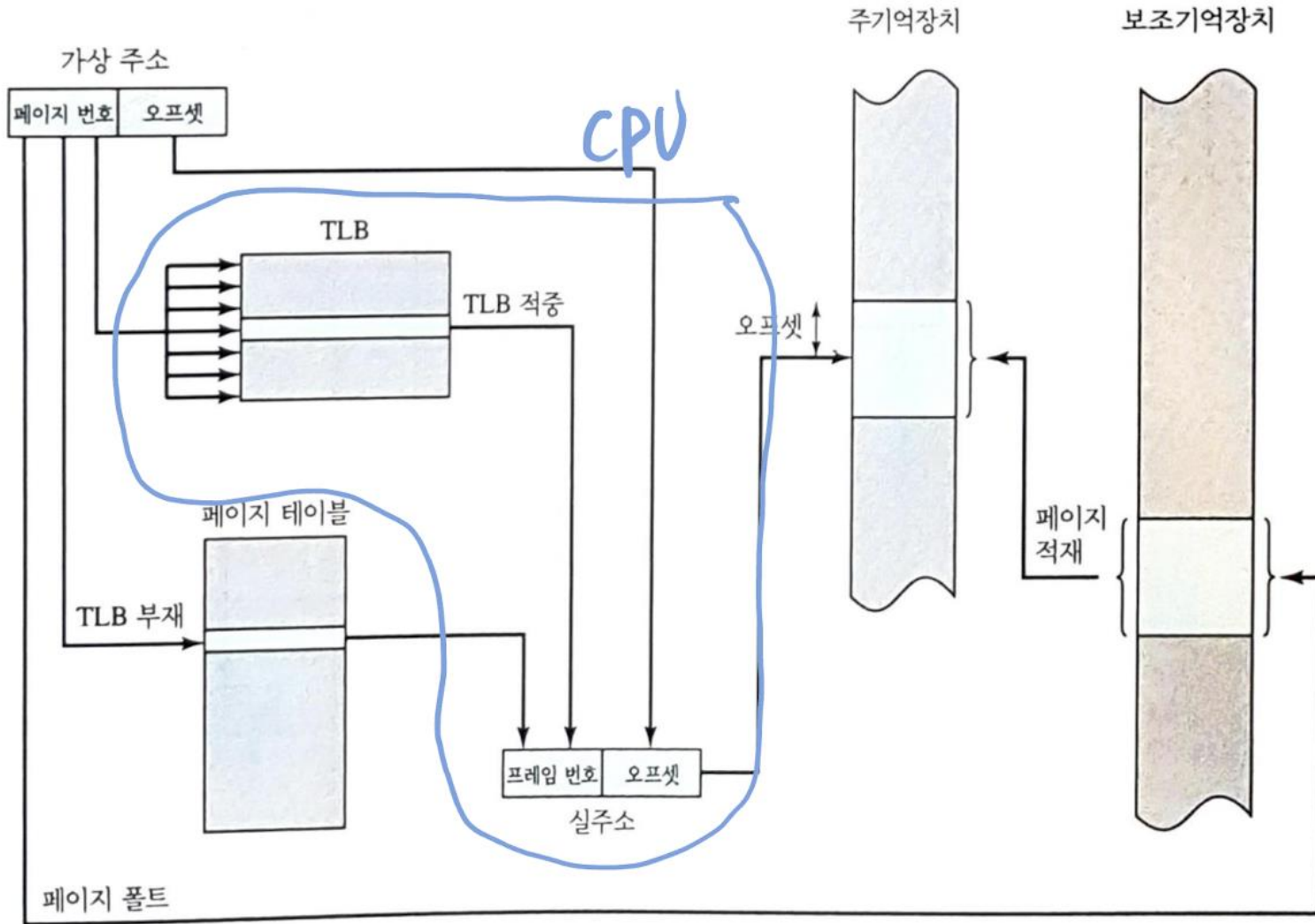
- 하드웨어 구성요소
- 논리주소를 물리주소로 바꾸기 위한 목적
- CPU와 RAM을 연결
- TLB는 MMU 내부의 고속 캐시
- 메모리 자원 효율적 사용
- 보안 향상
- 가상 메모리 지원

TLB 구조

V	Virtual Page Number	Physical Page Number	Dirty	Ref
1	10	101	0	1
1	50	101	0	0
0	-	-	-	-

- V: valid, 유효한 페이지인지
- Dirty: 메모리에 로드된 이후에 write 작업이 발생했는지
- Ref: 해당 페이지가 최근에 참조 되었는지 (페이지 교체 알고리즘에 사용)

CPU가 가상주소를 생성하면



TLB HIT

TLB에 존재하면 프레임 번호가 바로 추출되어 실주소를 구성

TLB MISS

존재하지 않으면 페이지 번호를 페이지 테이블 가서 인덱싱하여 찾는다.

1. 페이지 테이블 항목의 존재비트가 1이면 가져와서 동일하게 실주소 구성 -> TLB 갱신
2. 존재비트가 0이면 'Page Fault' 발생

그림 8.6 TLB(Translation Lookaside Buffer)의 사용

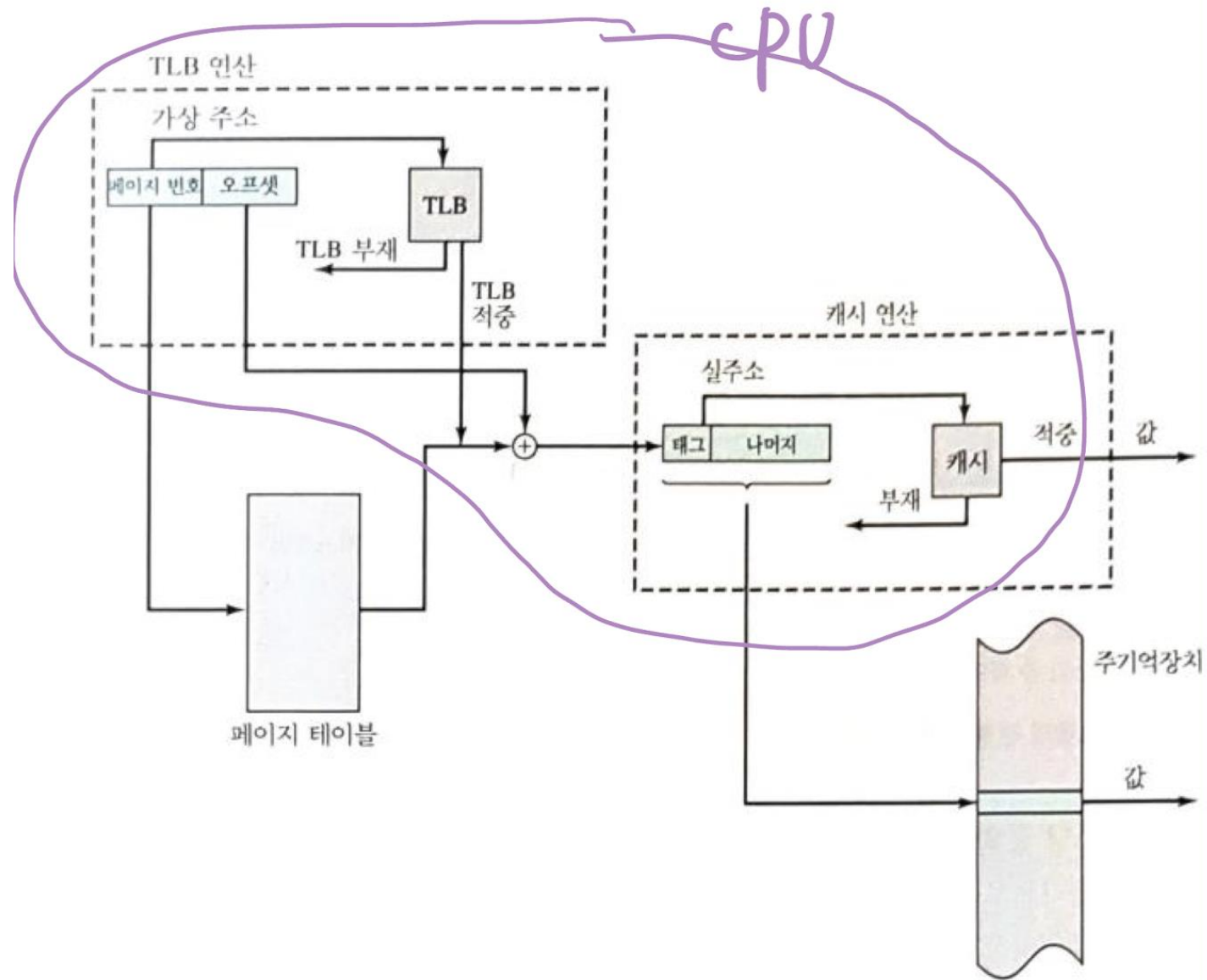


그림 8.9 TLB와 캐시의 동작

TLB의 연관 사상

Fully-associative 또는 Set-associative cache 구조 사용

- 각 엔트리의 태그와 입력 값이 일치하는지 “한 번에” 검사
- 고정된 위치가 아니기 때문에 hit rate 높음
- 병렬검사의 비용 (작은 TLB)

처리기에 연관사상을 할 수 있는 하드웨어(CAM)이 장착되어 페이지 번호와 일치하는 TLB항목을 찾음

↔ 페이지 테이블에서 PTE를 찾을 때 Direct Mapping

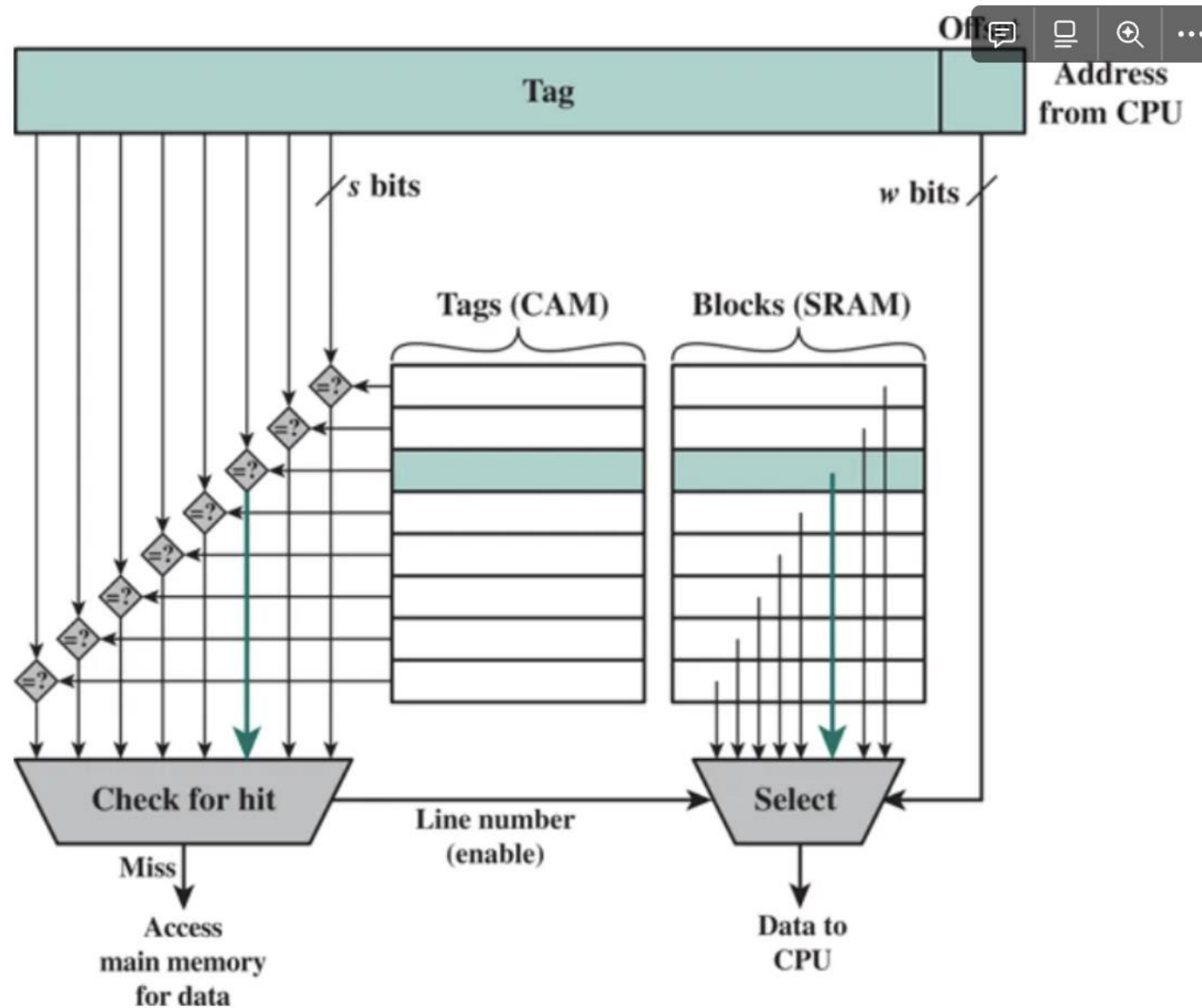


Figure 5.10 Fully Associative Cache Organization

TLB의 동기화

- 여러 기법

1. TLB Shutdown
2. Modified TLB Shutdown
3. Lazy devaluation
4. Read-locked TLBs
5. Memory-based TLBs
6. Validation

Context Switching과 TLB flush

TLB flush

CPU가 프로세스를 사용하면서 TLB에 저장한 (page, frame) 매핑 정보를 모두 지우는 과정이다. 주로 문맥 교환 과정에서 TLB flush가 발생한다.

왜 Context Switching에서 TLB flush?

1. 가상 주소 공간의 충돌 방지: 각 프로세스는 독립된 메모리 공간 사용
2. 주소 변환의 일관성 유지: 이전의 프로세스 매핑 정보를 TLB에 저장하여 사용

ASID

- 하드웨어 도움을 받아 TLB엔트리에 ASID를 추가하여 프로세스별 TLB엔트리를 구분하여 해결

V	Virtual Page Number	Physical Page Number	Dirty	Ref	ASID
1	10	101	0	1	1
1	50	101	0	0	2
0	-	-	-	-	-

멀티코어 TLB 동기화

TLB Shutdown

- IPI(Inter-processor Interrupt)를 통해 코어별 TLB 무효화 처리

캐시 일관성 프로토콜 연동

- MESI, TLB Shutdown 성능 향상

Lazy TLB Shutdown

- 무효화 작업을 지연시켜 오버헤드를 줄임

감사합니다